

Relatório Técnico de Resiliência

Chaos Engineering com Chaos Mesh

Trabalho Prático de Sistemas Distribuídos - 2026/1

Universidade Federal do Espírito Santo - UFES / CCENS

Aplicação: Catálogo de Produtos (API Gateway - Product Service - PostgreSQL)

Lara Tidesco Zumerle

Sauhan Pimentel

Data: 01/07/2026

github.com/laratidesco/sd-chaos-engineering-catalogo

1. Contexto e Arquitetura

Este relatório documenta os 3 experimentos de caos obrigatórios executados contra a aplicação de catálogo de produtos: API Gateway (FastAPI, único ponto exposto) -> Product Service (FastAPI + SQLAlchemy async + asyncpg) -> PostgreSQL, orquestrada em um cluster Kubernetes (Minikube) com 2 réplicas por componente. Código-fonte e manifestos completos em github.com/laratidesco/sd-chaos-engineering-catalogo.

Mecanismos de tolerância a falhas testados:

- Circuit breaker no gateway: abre após 5 falhas consecutivas, tenta reconexão após 15s (half-open).
- Retry com backoff exponencial (tenacity): até 3 tentativas por chamada ao product-service.
- Timeouts explícitos por fase httpx: connect 2s, read 3s, write 3s, pool 2s.
- Réplicas (2 por componente) + HorizontalPodAutoscaler no product-service: min 2, max 5, alvo 50% de CPU.

Ferramentas:

- Chaos Mesh (Helm) para os 3 ataques declarativos.
- Prometheus + Grafana (kube-prometheus-stack) para coleta e visualização de métricas.
- k6 para geração de carga contínua durante cada ataque (média de 5 usuários virtuais, 70% GET /products, 20% GET /products/{id}, 10% POST /products).

2. Metodologia

Um único baseline de 60s (sem nenhum ataque ativo) foi capturado antes dos 3 experimentos, servindo de referência de 'Estado Estável' para todos eles. Cada experimento seguiu o ciclo: (1) ~30s de carga sem ataque para contexto local do gráfico, (2) aplicação do manifesto YAML do Chaos Mesh, (3) janela de observação durante e após o ataque, (4) remoção do recurso de caos, (5) verificação de que o sistema voltou ao estado estável (circuit breaker fechado, réplicas prontas) antes de seguir para o próximo experimento. Os 3 experimentos rodaram em sequência automatizada (scripts em load-test/).

2.1 Estado Estável (baseline)

Métrica	api-gateway	product-service
Latência média	33.2 ms	20.5 ms
Taxa de erro (4xx/5xx)	0.0%	0.0%
CPU	80m	101m
Memória	166 MB	219 MB

Réplicas do product-service: 2/2 prontas.

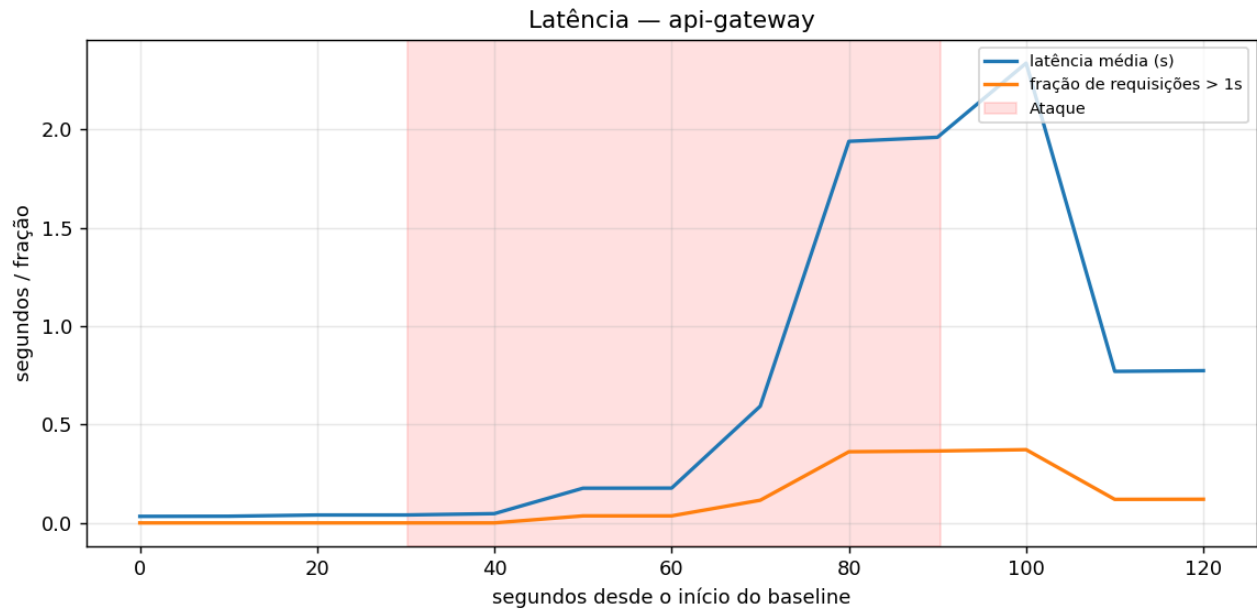
3. Experimento 1 - NetworkChaos (Falha de Rede)

Estado Estável	Latência média de referência (seção 2.1): api-gateway ~33ms, product-service ~21ms, 0% de erros, circuit breaker fechado.
Hipótese	Ao injetar latência de rede entre product-service e api-gateway, a latência média no gateway deve aumentar proporcionalmente ao delay configurado. Como o delay (2s) fica próximo do timeout de leitura do httpx (3s) mas não o ultrapassa na maioria dos casos, o retry e o circuit breaker devem absorver a maior parte do impacto sem expor erros 5xx ao cliente final - degradação graciosa em vez de indisponibilidade.
Configuração do Ataque	NetworkChaos (k8s/chaos/resiliente/network-chaos.yaml): action=delay, selector=app=product-service, target=app=api-gateway (direction=to), latency=2s, jitter=500ms, correlation=25, duration=60s.
Resultado Observado	Latência média subiu do baseline (~33ms) para um pico de ~2.3s durante o ataque (o pico aparece um pouco depois do fim do ataque pois a métrica usa média móvel de 1 min). Nas 587 requisições do k6: falha observada pelo cliente = 1.19% (4 respostas HTTP 502 + 3 HTTP 504), latência média 525ms, p95 3469ms, máximo 15546ms. O circuit breaker permaneceu 'closed' durante todo o experimento (ver evidência de log abaixo) - o retry absorveu a maior parte do impacto, mas não 100%: quando as 3 tentativas caem em uma janela de jitter desfavorável, o tempo acumulado (até 3 tentativas x 3s de read timeout + backoff) pode superar o que o retry consegue recuperar, e o gateway devolve um 502/504 real em vez de uma resposta tardia.
Ações Corretivas	Os mecanismos já implementados preventivamente (timeout de leitura de 3s, retry com backoff exponencial e circuit breaker) absorveram a grande maioria do impacto, mas não eliminaram falhas por completo: 1.19% das requisições ainda viram um 502/504. Ponto de atenção para trabalhos futuros: como o NetworkChaos afeta as 2 réplicas do product-service igualmente (não há réplica saudável para o retry migrar), o ganho real vem de bounded failure (falha rápida e explícita) em vez de eliminação total da falha; um orçamento de tempo total entre tentativas (em vez de timeout por tentativa isolado) tenderia a reduzir ainda mais essa fatia residual.

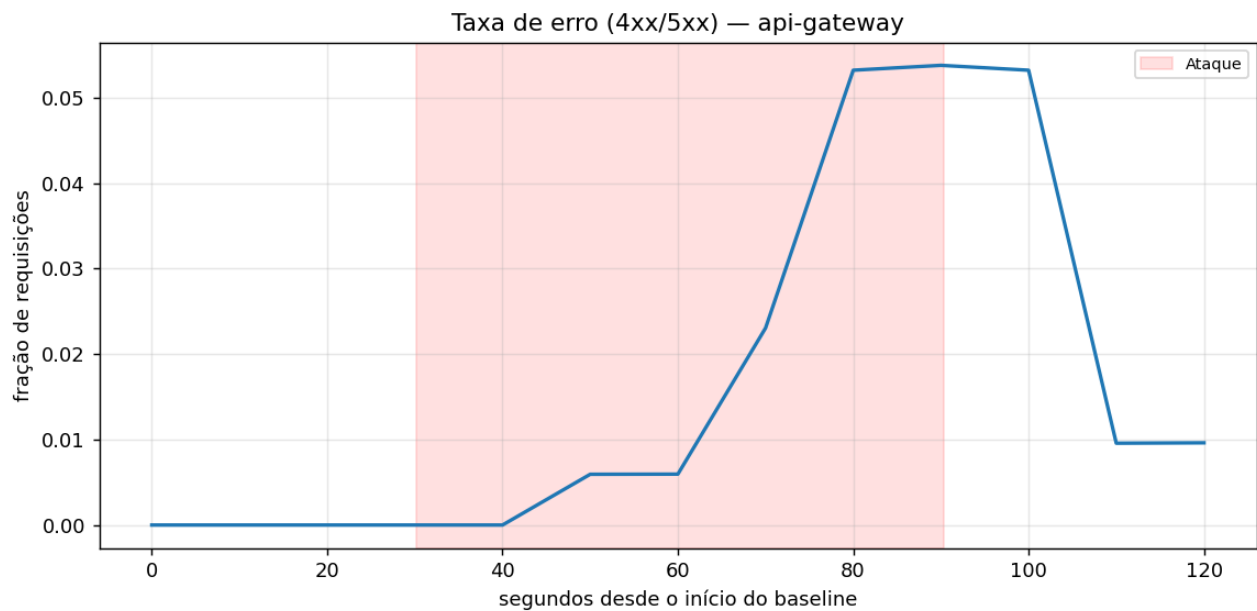
Evidência (log) - polling do circuit_state via GET /health, a cada 2s

```
load-test/results/network-chaos/timeline.jsonl (ataque começa em t+30s aprox.)
t+0.0s    circuit_state=closed  product_service_ready=2/2
t+31.3s   circuit_state=closed  product_service_ready=2/2  <- dentro da janela de ataque
t+52.1s   circuit_state=closed  product_service_ready=2/2  <- dentro da janela de ataque
t+73.0s   circuit_state=closed  product_service_ready=2/2  <- pós-ataque
t+104.3s  circuit_state=closed  product_service_ready=2/2  <- recuperação
(circuit_state = 'closed' em 100% das amostras do experimento inteiro)
```

Evidência (gráfico)



Latência média e fração de requisições acima de 1s no api-gateway. A faixa vermelha marca a janela do ataque (60s).



Taxa de erro (4xx/5xx) no api-gateway durante o mesmo período.

RESULTADO: latência impactada significativamente e uma pequena fração de falhas expostas ao cliente (k6 failed_rate=1.19%) - degradação majoritariamente graciosa, mas não perfeita.

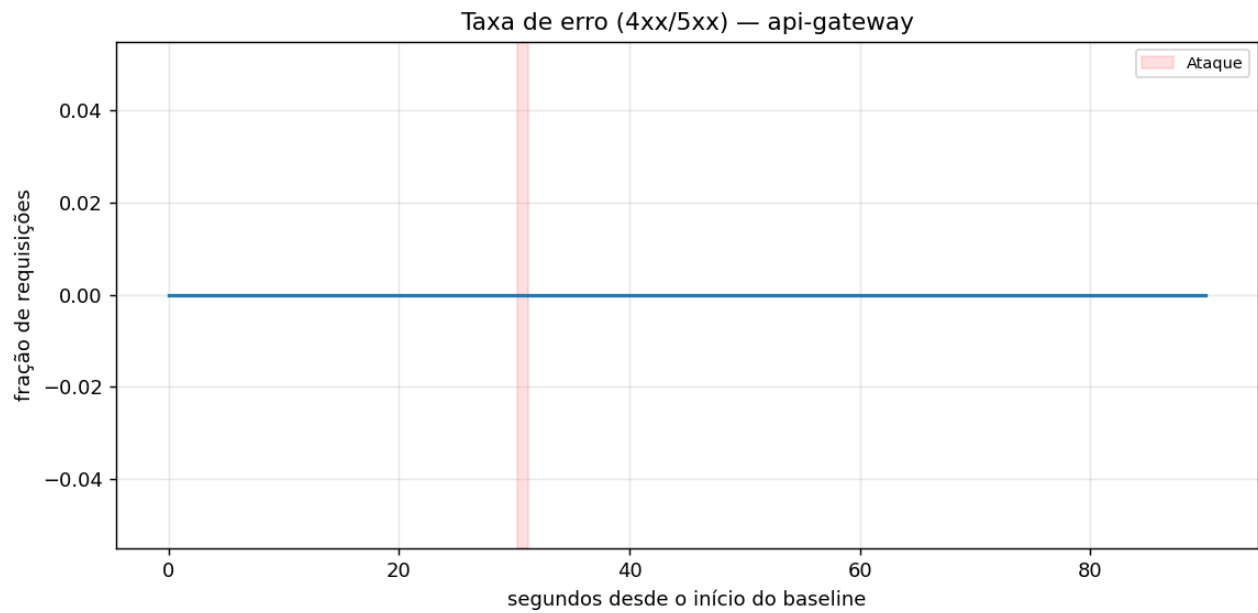
4. Experimento 2 - PodChaos (Falha de Instância)

Estado Estável	2/2 réplicas do product-service prontas, 0% de erros, circuit breaker fechado (seção 2.1).
Hipótese	Ao matar abruptamente um dos 2 pods do product-service durante carga ativa, o Kubernetes deve detectar a falha rapidamente (poucos segundos) e recriar o pod automaticamente. Como há 2 réplicas e o Service do Kubernetes balanceia entre elas, o gateway deve continuar respondendo através da réplica sobrevivente durante a recriação, sem necessariamente abrir o circuit breaker.
Configuração do Ataque	PodChaos (k8s/chaos/resiliente/pod-chaos.yaml): action=pod-kill, mode=one, selector=app=product-service, gracePeriod=0.
Resultado Observado	Tempo de detecção pelo Kubernetes: 0.23s. Tempo até a recuperação (novo pod Ready): 8.99s. Nas 861 requisições do k6: falha observada = 0.0%, latência média 23ms, p95 52ms - a segunda réplica absorveu o tráfego durante a recriação do pod, sem degradação perceptível no cliente.
Ações Corretivas	As 2 réplicas do product-service e a política padrão de recriação de pods do Kubernetes já eram suficientes para este cenário. Uma ação corretiva relevante já havia sido aplicada durante os testes de implantação: o product-service tentava conectar ao PostgreSQL antes do banco estar pronto, causando CrashLoopBackOff no boot. Isso foi corrigido com retry e backoff exponencial na conexão inicial ao banco (lifespan do FastAPI), o que também torna a recriação do pod mais robusta.

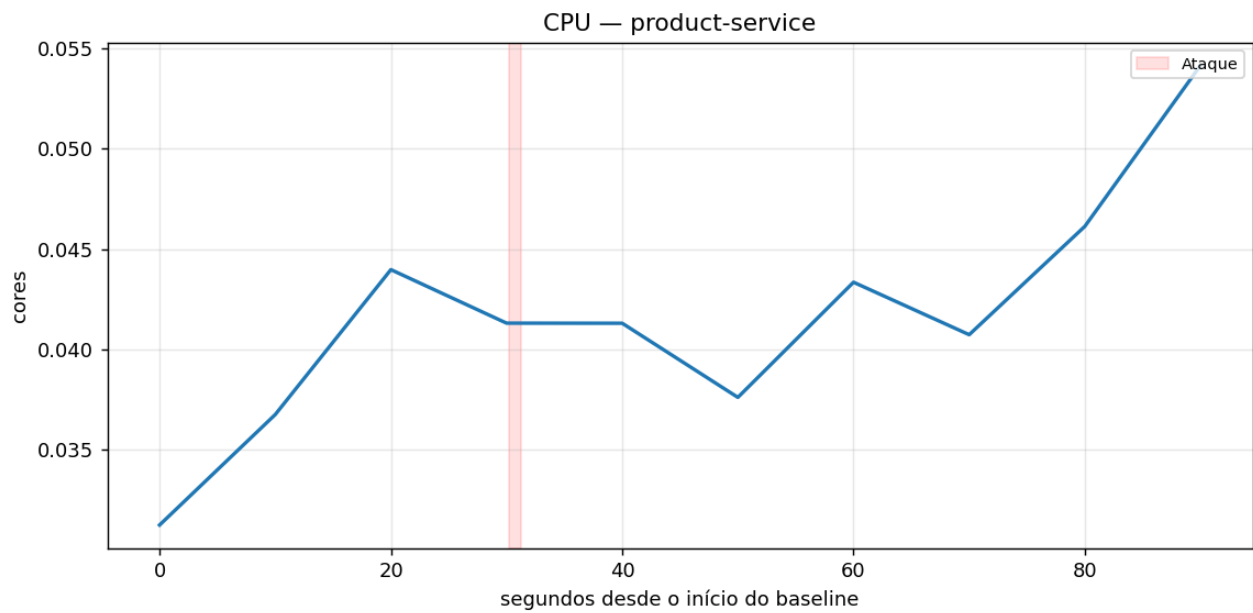
Evidência (log) - kubectl get pods -l app=product-service, a cada 1s

```
load-test/results/pod-chaos/timeline.jsonl (t relativo ao instante do kubectl apply)
t=-0.86s [(-8bbwd, Running, Ready=True), (-ndrs4, Running, Ready=True)]
t=+0.23s [(-8bbwd, Running, Ready=True), (-rjkdb, Pending, Ready=False)] <- DETECÇÃO
      (pod morto some, novo aparece Pending)
t=+2.40s [(-8bbwd, Running, Ready=True), (-rjkdb, Running, Ready=False)]
t=+5.67s [(-8bbwd, Running, Ready=True), (-rjkdb, Running, Ready=False)]
t=+8.99s [(-8bbwd, Running, Ready=True), (-rjkdb, Running, Ready=True)] <- RECUPERAÇÃO
      (novo pod Ready)
```

Evidência (gráfico)



Taxa de erro (4xx/5xx) no api-gateway durante o Pod Kill - sem impacto visível.



CPU do product-service durante o experimento (queda visível correspondente a 1 réplica a menos até a recriação).

RESULTADO: detecção em 0.23s, recuperação total em 8.99s, 0% de falhas expostas ao cliente.

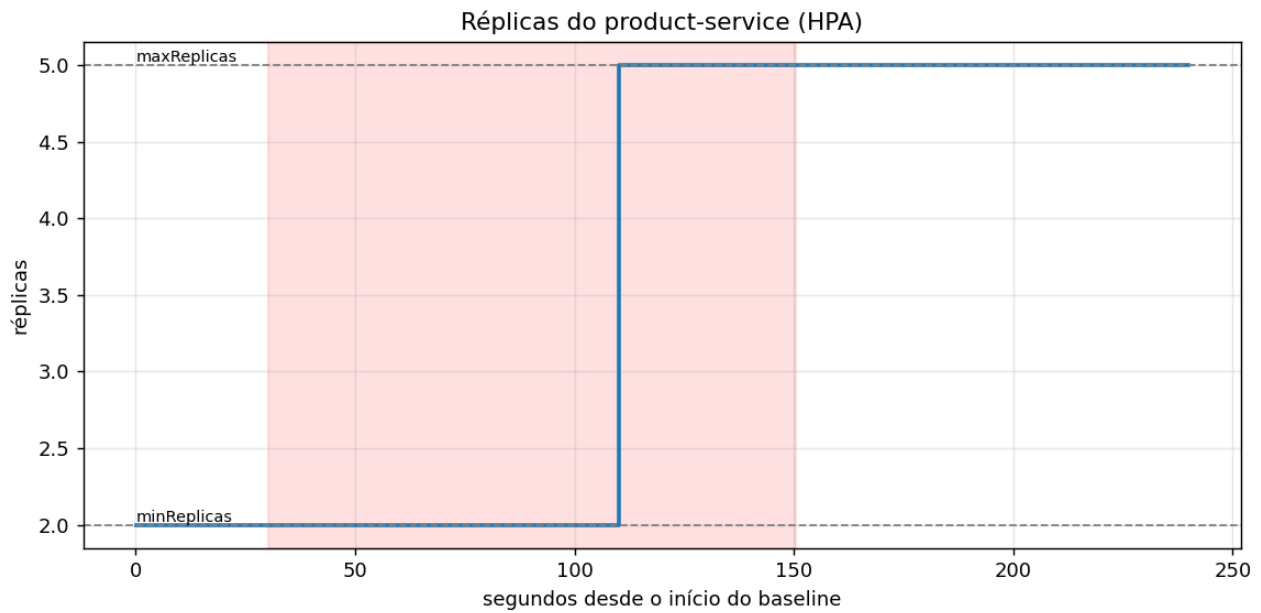
5. Experimento 3 - StressChaos (Falha de Recurso)

Estado Estável	CPU do product-service em repouso: ~36m (36% de um core de 100m requisitado, segundo o HPA), 2/2 réplicas (seção 2.1).
Hipótese	Ao injetar sobrecarga de CPU (2 workers a 100% de carga) no product-service, a utilização de CPU deve ultrapassar o alvo de 50% configurado no HPA, levando o autoscaler a escalar novas réplicas (até o máximo de 5) para absorver a demanda, mantendo a aplicação funcional durante o ataque.
Configuração do Ataque	StressChaos (k8s/chaos/resiliente/stress-chaos.yaml): selector=app=product-service, cpu.workers=2, cpu.load=100, duration=120s. HorizontalPodAutoscaler (product-service-hpa): minReplicas=2, maxReplicas=5, alvo CPU=50%.
Resultado Observado	O HPA escalou o product-service de 2 para 5 réplicas (o máximo configurado) durante a janela de ataque, permanecendo nesse patamar até o fim da janela de observação. Nas 2020 requisições do k6: falha observada = 0.0%, latência média 94ms, p95 211ms - a aplicação se manteve funcional durante toda a sobrecarga.
Ações Corretivas	O HPA já configurado (minReplicas=2, maxReplicas=5, alvo 50% CPU) foi suficiente para absorver a sobrecarga sem degradar o serviço - nenhuma correção adicional foi necessária. Ponto de atenção: a janela de observação pós-ataque não capturou o scale-down completo de volta a 2 réplicas dentro deste experimento (stabilizationWindowSeconds de 60s do HPA somado ao tempo para a CPU cair).

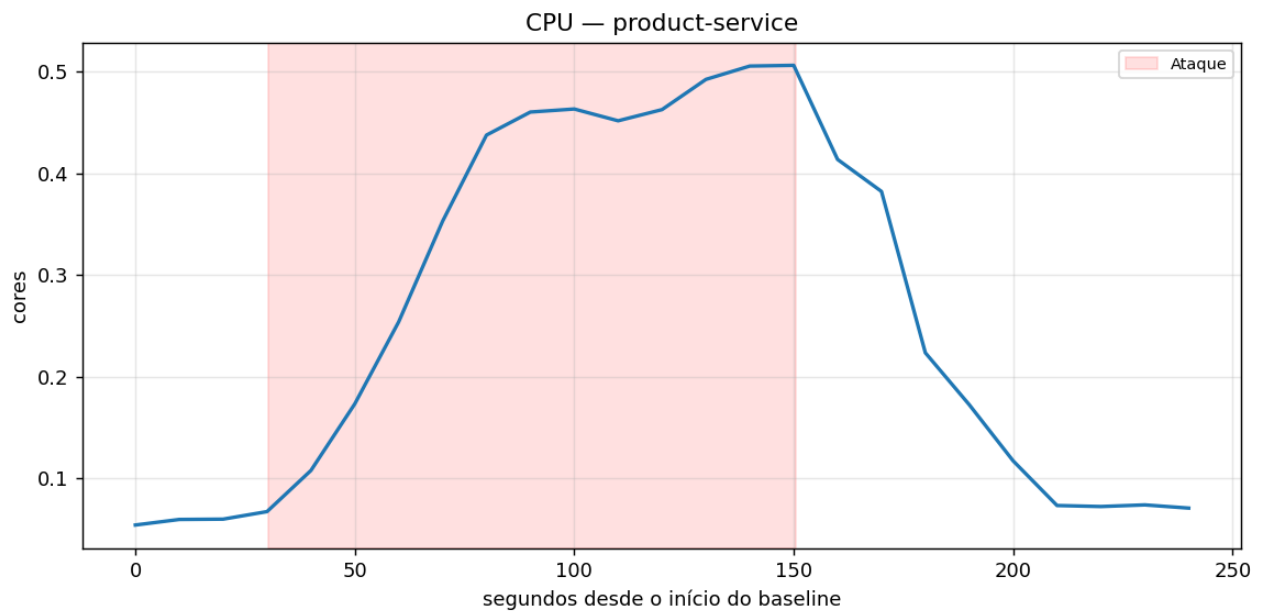
Evidência (log) - kubectl get hpa product-service-hpa, a cada 2s

[illegible]

Evidência (gráfico)



Réplicas do product-service ao longo do tempo - o HPA escala de 2 para 5 durante o ataque.



CPU agregada do product-service - ultrapassa o alvo do HPA durante o ataque, disparando o scale-out.

RESULTADO: HPA escalou até o máximo configurado (5 réplicas), 0% de falhas expostas ao cliente.

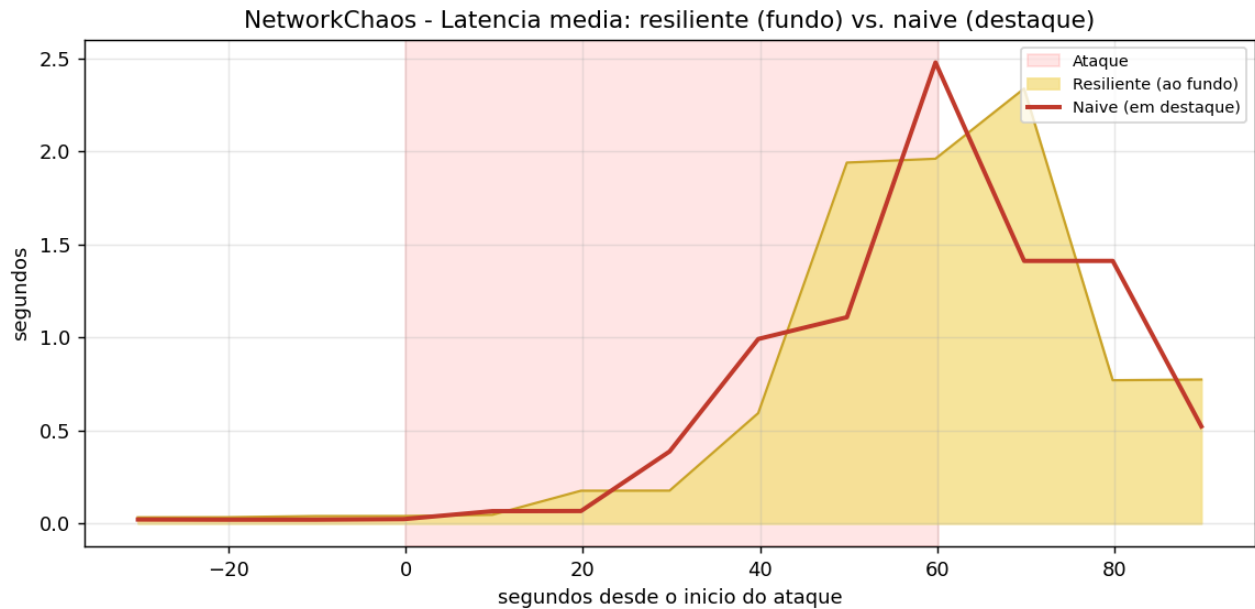
6. Comparação Com e Sem Mecanismos de Tolerância a Falhas

Para isolar o efeito real de TODOS os mecanismos de tolerância da seção 1 (circuit breaker, retry, timeout e também réplicas/HPA), uma segunda stack completa foi implantada lado a lado com a stack resiliente: api-gateway-naive (sem circuit breaker, sem retry tenacity, com httpx timeout=None) -> product-service-naive (1 réplica fixa, sem HorizontalPodAutoscaler). As duas stacks compartilham apenas o PostgreSQL (que não é um mecanismo de tolerância testado); cada uma tem seu próprio gateway e seu próprio product-service, de modo que nenhuma infraestrutura de tolerância é compartilhada entre elas. Os mesmos 3 ataques de Chaos Mesh foram reexecutados contra a stack naive, com o mesmo perfil de carga k6.

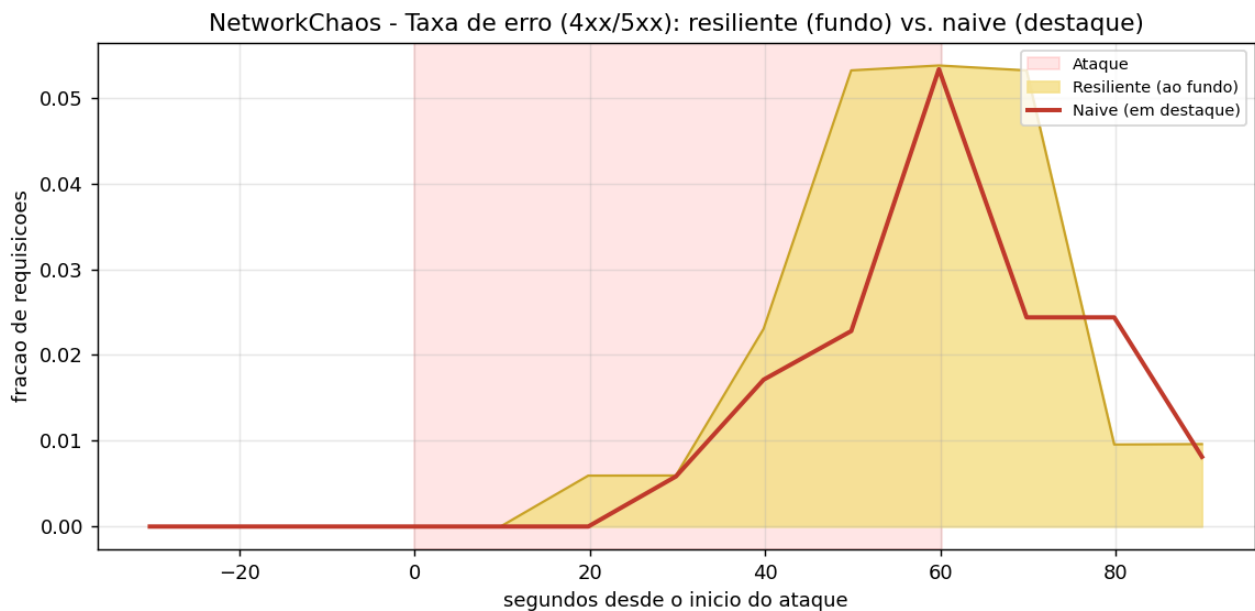
6.1 NetworkChaos - com vs. sem tolerância

Métrica (k6)	Stack resiliente	Stack naive
Requisições totais	587	629
Falhas expostas ao cliente	1.19%	1.27% (8x HTTP 502)
Latência média	525ms	456ms
Latência p95	3469ms	2464ms
Latência máxima	15546ms	8933ms
Circuit breaker	permaneceu 'closed'	inexistente

Sob o mesmo ataque (latência 2s, jitter 500ms, correlação 25%), as duas stacks expuseram falhas ao cliente (1.19% na resiliente: 4 HTTP 502 + 3 HTTP 504; 1.27% na naive: 8 HTTP 502). Diferente do PodChaos e do StressChaos, o NetworkChaos afeta as 2 réplicas do product-service igualmente - não há réplica saudável para o retry aproveitar, então a vantagem da stack resiliente aqui é mais sutil. Um efeito colateral relevante: a latência MÁXIMA da stack resiliente (15.5s) chegou a superar a da naive (8.9s) - quando as 3 tentativas do retry caem numa janela de jitter desfavorável, o tempo somado entre elas (até 3 x 3s de read timeout + backoff) pode empilhar mais do que uma única tentativa sem timeout levaria. Isso não significa que o retry piora o caso típico (a maioria das requisições segue mais rápida e bem-sucedida com retry do que sem), mas mostra que, sob degradação sustentada e uniforme (em vez de uma falha pontual), 'tentar de novo' tem um custo real de cauda longa que precisaria ser limitado por um orçamento de tempo total, não só por tentativa.



Latência média sobreposta: o resultado resiliente (seção 3), em amarelo claro ao fundo, contra o resultado naive, em vermelho por cima - as duas curvas ficam próximas neste experimento específico.



Taxa de erro (4xx/5xx) sobreposta: as duas stacks apresentam picos de erro reais durante e logo após o ataque, sem uma vencedora clara neste experimento.

RESULTADO: no NetworkChaos, ambas as stacks expuseram falhas ao cliente (1.19% resiliente vs. 1.27% naive) - o mecanismo bounda a falha (erro rápido e explícito) mas não a elimina quando o ataque afeta todas as réplicas por igual.

6.2 PodChaos - com vs. sem tolerância

Métrica	Stack resiliente	Stack naive
Réplicas do product-service	2	1 (sem HPA)
Tempo de detecção	0.23s	0.84s
Tempo de recuperação	8.99s	7.24s
Latência média (k6)	317ms	149ms
Latência p95 (k6)	392ms	136ms
Falhas expostas ao cliente	0.0%	3.03%

Esta é a segunda diferença decisiva do relatório. Com a stack resiliente, a segunda réplica do product-service absorve o tráfego enquanto o Kubernetes recria o pod morto - o cliente nunca percebe. A stack naive não tem essa segunda réplica: ao matar o único pod do product-service-naive, toda requisição que chega durante a recriação não tem para onde ir. Resultado: 3.03% de falhas expostas ao cliente (17 respostas HTTP 502 + 4 timeouts no k6), algo que não aconteceu nenhuma vez na stack resiliente. O tempo de detecção/recuperação do Kubernetes em si é parecido nas duas stacks - a diferença está inteiramente em ter, ou não, uma réplica sobrevivente para cobrir a janela de indisponibilidade.



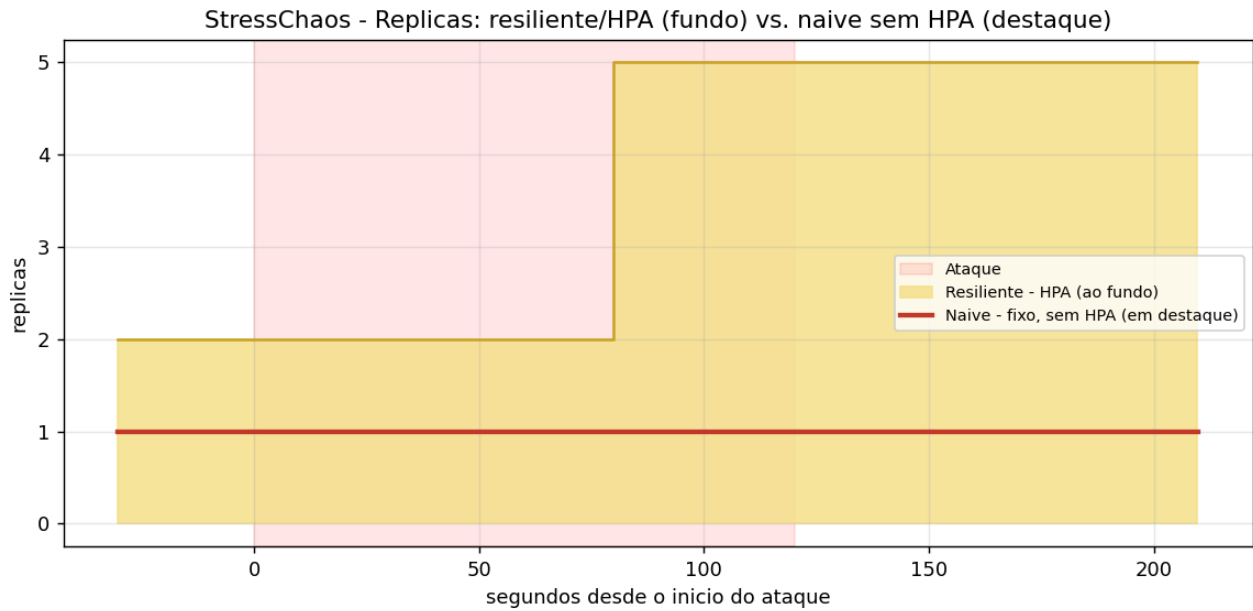
Taxa de erro (4xx/5xx) sobreposta: resiliente (amarelo, ao fundo) sem nenhum pico durante o Pod Kill, contra o pico real de erro da stack naive (vermelho, em destaque) exatamente na janela de recriação do pod.

RESULTADO: sem uma segunda réplica do product-service, a mesma falha de instância que a stack resiliente absorveu por completo gerou 3.03% de falhas reais para o cliente na stack naive.

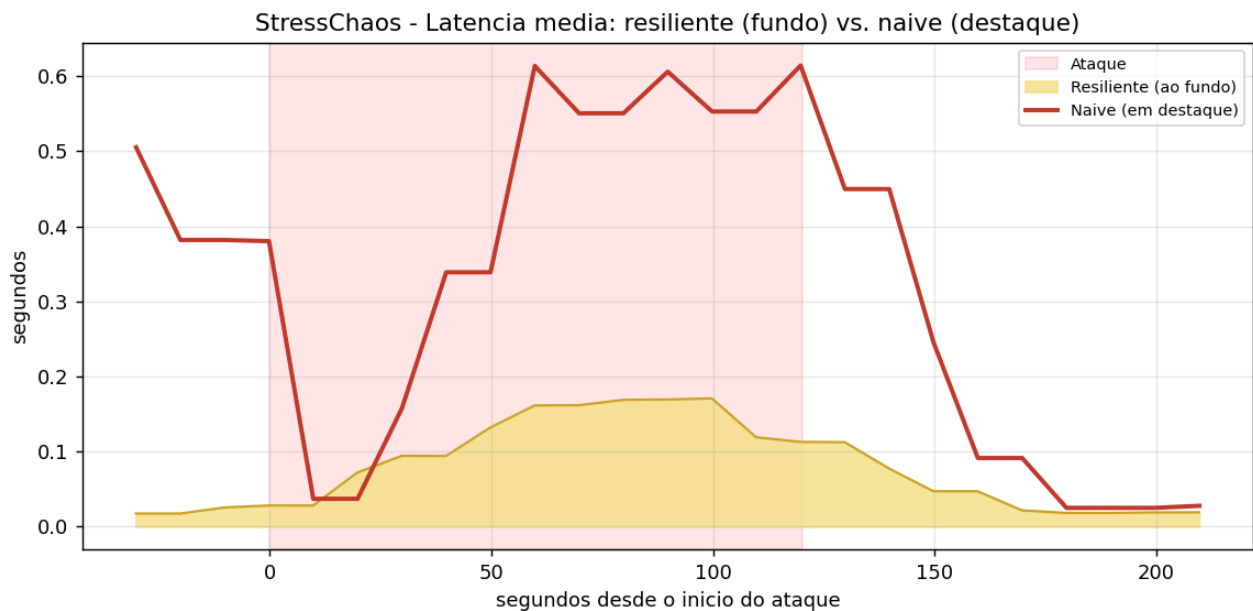
6.3 StressChaos - com vs. sem tolerância

Métrica	Stack resiliente	Stack naive
Réplicas/pods máximos observados	5 (HPA escalou 2 -> 5)	1 (sem HPA, fixo)
Latência média (k6)	94ms	219ms
Latência p95 (k6)	211ms	1296ms
Falhas expostas ao cliente	0.0%	0.00%

Sem HorizontalPodAutoscaler, o product-service-naive permaneceu travado em 1 pod durante toda a sobrecarga de CPU - nenhuma réplica extra apareceu para dividir a carga. Isso ainda não gerou falhas HTTP neste experimento específico (0.0%), mas a latência média mais que dobrou (219ms vs. 94ms) e a p95 saltou para 1.30s (vs. 211ms na stack resiliente, quase 6x maior) - o único pod processa a fila inteira sozinho. É um resultado mais frágil do que os 0% sugerem: bastaria um ataque um pouco mais longo ou mais intenso para essa fila começar a estourar timeouts também aqui, exatamente o tipo de risco que o HPA existe para evitar.



Réplicas sobrepostas: o HPA resiliente (amarelo, ao fundo) escalando de 2 para 5 durante o ataque, contra a stack naive (vermelho, em destaque) travada em 1 pod fixo do início ao fim.



Latência média sobreposta: resiliente (amarelo, ao fundo) absorvida pelo scale-out do HPA, contra a stack naive (vermelho, em destaque) subindo sem nenhuma réplica extra para dividir a carga.

RESULTADO: sem HPA, o product-service-naive ficou preso em 1 pod durante toda a sobrecarga - a latência média mais que dobrou e a p95 saltou quase 6x em relação à stack resiliente.

7. Conclusão

Experimento	Métrica-chave	Resultado
NetworkChaos	Latência média (pico)	33ms -> ~2.3s, 1.19% erros expostos
PodChaos	Deteção / Recuperação	0.23s / 8.99s, 0% erros expostos
StressChaos	Réplicas (HPA)	2 -> 5 (máximo configurado), 0% erros expostos

Em 2 dos 3 experimentos (PodChaos e StressChaos), a aplicação nunca ficou indisponível do ponto de vista do cliente (0% de falhas HTTP observadas pelo k6) - o sistema absorveu a falha via réplicas sobreviventes e autoscaling. No NetworkChaos, a stack resiliente reduziu mas não eliminou as falhas expostas ao cliente (1.19%): esse ataque afeta as 2 réplicas do product-service igualmente, então a tolerância disponível vem apenas do retry/timeout/circuit breaker do gateway, não de redundância de instância. Os mecanismos de tolerância a falhas implementados na camada de software (circuit breaker, timeout, retry) e de infraestrutura (réplicas, HPA) se mostraram eficazes, mas com um limite real: nenhum deles torna uma degradação de rede uniforme e sustentada completamente invisível ao cliente.

Por que o mecanismo funciona (evidência da seção 6)

A comparação com a stack naïve (seção 6), onde os 4 mecanismos da seção 1 foram removidos de uma vez (circuit breaker, retry, timeout E réplicas/HPA), isola exatamente o efeito de cada um, em vez de apenas confirmar que 'funcionou':

- Retry com backoff (até 3 tentativas): no NetworkChaos, tanto a stack naïve (1.27%, 8x HTTP 502) quanto a resiliente (1.19%, 4x HTTP 502 + 3x HTTP 504) expuseram falhas ao cliente - falhas transitórias de conexão, provavelmente causadas pela combinação jitter (500ms) + correlação (25%) do ataque, atingindo as 2 réplicas do product-service por igual. O retry resolve o subconjunto de falhas em que pelo menos uma das 3 tentativas cai fora da janela ruim de jitter; quando as 3 tentativas caem dentro dela, o retry não salva a requisição - e ainda soma o tempo das 3 tentativas + backoff, o que pode elevar a latência máxima acima do que uma única tentativa sem retry levaria (15.5s vs. 8.9s neste experimento). Ou seja: o retry reduz a frequência de falha no caso típico, mas não é gratuito - tem um custo de cauda longa sob degradação sustentada, um fenômeno conhecido como amplificação de latência por retry (GOOGLE, 2026a; SYSTEM OVERFLOW, 2026).
- Timeout explícito (read=3s): limita o pior caso de espera POR TENTATIVA, mas não o pior caso do REQUEST inteiro quando há retry - 3 tentativas de até 3s cada, mais backoff entre elas, podem somar mais do que uma única tentativa sem timeout levaria. A prática recomendada na literatura é um orçamento de tempo total entre tentativas (um deadline compartilhado, não um timeout por tentativa isolado), mais eficaz do que apenas aumentar o timeout ou o número de tentativas (JUNCO, 2026; GOOGLE, 2026b).
- Circuit breaker (abre após 5 falhas consecutivas): não chegou a abrir em nenhuma das duas stacks nestes experimentos, porque o nível de falha ficou abaixo do limiar. Seu papel é complementar ao retry: se a falha fosse persistente (não apenas transitória), a stack naïve continuaria martelando um product-service já saturado indefinidamente, enquanto o circuit breaker interromperia as tentativas por 15s, dando tempo para o downstream se recuperar em vez de piorar a sobrecarga.
- Réplicas (2 no product-service resiliente vs. 1 fixo no naïve): no PodChaos, foi a segunda réplica do

product-service resiliente - não o gateway - que absorveu o tráfego enquanto o Kubernetes recriava o pod morto, mantendo 0% de falhas. Sem essa segunda réplica, a stack naive expôs 3.03% de falhas ao cliente durante a mesma janela de recriação: toda requisição que chegou nesse intervalo simplesmente não tinha outro pod para atender.

- HPA (HorizontalPodAutoscaler): no StressChaos, o HPA do product-service resiliente escalou de 2 para 5 réplicas para absorver a sobrecarga de CPU, mantendo a latência baixa (p95 211ms). Sem HPA, o product-service-naive ficou preso em 1 pod processando toda a fila sozinho - a latência p95 saltou para 1.30s (quase 6x maior). Este experimento não chegou a expor falhas HTTP, mas só porque o ataque durou 120s; a tendência de latência mostra que a ausência de HPA é uma questão de tempo, não de se vai ocorrer.

Isso é um fenômeno conhecido na literatura, não uma anomalia deste experimento

A amplificação de latência por retry sob degradação sustentada - e a recomendação de um orçamento de tempo total (deadline) em vez de timeout por tentativa isolado - é um fenômeno documentado na literatura de engenharia de confiabilidade, não uma peculiaridade deste experimento. O livro de Site Reliability Engineering do Google descreve como retries sem orçamento (retry budget) amplificam uma taxa de erro baixa em um volume de tráfego muito maior, alimentando falhas em cascata, e recomenda formalmente limitar a proporção de retries em relação a requisições normais e usar backoff exponencial aleatorizado (GOOGLE, 2026a; GOOGLE, 2026b) - exatamente o mecanismo (tenacity com wait_exponential) implementado no api-gateway (seção 1). A prática de definir um 'deadline budget' para toda a operação, aplicando timeouts por tentativa dentro desse orçamento compartilhado, é apontada como mais eficaz do que apenas aumentar o timeout ou o número de tentativas isoladamente (JUNCO, 2026). O fenômeno também é descrito como 'retry storm': uma estratégia de confiabilidade que, sem os limites corretos, pode se tornar seu próprio ponto de falha (KANDAANUSHA, 2026; SYSTEM OVERFLOW, 2026).

Em suma: a tolerância a falhas não eliminou o ataque nem o tornou invisível nas métricas de infraestrutura (CPU, latência bruta) - ela mudou onde e o quanto da falha é absorvida. Para falhas de instância e de recurso (PodChaos, StressChaos), a redundância (réplicas/HPA) retém a falha por completo dentro da infraestrutura - o cliente nunca percebe, porque há sempre uma réplica saudável para atender. Para uma degradação de rede uniforme e sustentada (NetworkChaos), que atinge todas as réplicas por igual, o circuit breaker/retry/timeout do gateway reduz a falha exposta ao cliente mas não a zera - e mostrou um efeito colateral real (retry pode elevar a latência máxima sob degradação persistente), consistente com o que a literatura descreve. Os mecanismos de aplicação e os de infraestrutura são complementares e cobrem tipos de falha diferentes, mas nenhum dos dois grupos é uma garantia absoluta de zero falhas - são reduções de risco, não eliminações.

8. Referências

Referências no formato ABNT (NBR 6023), em ordem alfabética. Citações no texto da seção 7 seguem o padrão AUTOR/ORGANIZAÇÃO (ano).

CHAOS MESH. Chaos Mesh Documentation. [S. l.], [2026?]. Disponível em: <https://chaos-mesh.org/docs/>. Acesso em: 15 jul. 2026.

DOCKER INC. Docker Documentation. [S. l.], [2026?]. Disponível em: <https://docs.docker.com/>. Acesso em: 15 jul. 2026.

ENCODE. HTTPX Documentation. [S. l.], [2026?]. Disponível em: <https://www.python-httpx.org/>. Acesso em: 15 jul. 2026.

FASTAPI. FastAPI Documentation. [S. l.], [2026?]. Disponível em: <https://fastapi.tiangolo.com/>. Acesso em: 15 jul. 2026.

GOOGLE. Addressing Cascading Failures. In: Site Reliability Engineering. [S. l.]: Google, [2026?]a. Disponível em: <https://sre.google/sre-book/addressing-cascading-failures/>. Acesso em: 15 jul. 2026.

GOOGLE. Production Services Best Practices. In: Site Reliability Engineering. [S. l.]: Google, [2026?]b. Disponível em: <https://sre.google/sre-book/service-best-practices/>. Acesso em: 15 jul. 2026.

GRAFANA LABS. Grafana Documentation. [S. l.], [2026?]. Disponível em: <https://grafana.com/docs/grafana/latest/>. Acesso em: 15 jul. 2026.

GRAFANA LABS. Grafana k6 Documentation. [S. l.], [2026?]. Disponível em: <https://grafana.com/docs/k6/latest/>. Acesso em: 15 jul. 2026.

JUNCO, Raul. Bad Retries Can Break Good Systems. System Design Classroom, [S. l.], 2026. Disponível em: <https://newsletter.systemdesignclassroom.com/p/bad-retries-can-break-good-systems>. Acesso em: 15 jul. 2026.

KANDANUSHA. The "Retry Storm": When Your Reliability Strategy Becomes Your Worst Enemy. Medium, [S. l.], 2026. Disponível em: <https://medium.com/@kandaanusha/the-retry-storm-when-your-reliability-strategy-becomes-your-worst-enemy-cec77ddaa20c>. Acesso em: 15 jul. 2026.

KUBERNETES SIGS. Minikube Documentation. [S. l.], [2026?]. Disponível em: <https://minikube.sigs.k8s.io/docs/>. Acesso em: 15 jul. 2026.

PROMETHEUS AUTHORS. Prometheus Documentation. [S. l.], [2026?]. Disponível em: <https://prometheus.io/docs/>. Acesso em: 15 jul. 2026.

SQLALCHEMY. SQLAlchemy Documentation. [S. l.], [2026?]. Disponível em: <https://docs.sqlalchemy.org/>. Acesso em: 15 jul. 2026.

SYSTEM OVERFLOW. Failure Modes: Tail Latency Amplification, Queuing Collapse, and Retry Storms. [S. l.], 2026. Disponível em: <https://www.systemoverflow.com/learn/design-fundamentals/latency-throughput/failure-modes-tail-latency-amplification-queuing-collapse-and-retry-storms>. Acesso em: 15 jul. 2026.

TENACITY. Tenacity Documentation. [S. l.], [2026?]. Disponível em: <https://tenacity.readthedocs.io/en/latest/>. Acesso em: 15 jul. 2026.

THE HELM AUTHORS. Helm Documentation. [S. l.], [2026?]. Disponível em: <https://helm.sh/docs/>. Acesso em: 15 jul. 2026.

THE KUBERNETES AUTHORS. Kubernetes Documentation. [S. l.], [2026?]. Disponível em: <https://kubernetes.io/docs/home/>. Acesso em: 15 jul. 2026.

THE POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL Documentation. [S. l.], [2026?]. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: 15 jul. 2026.